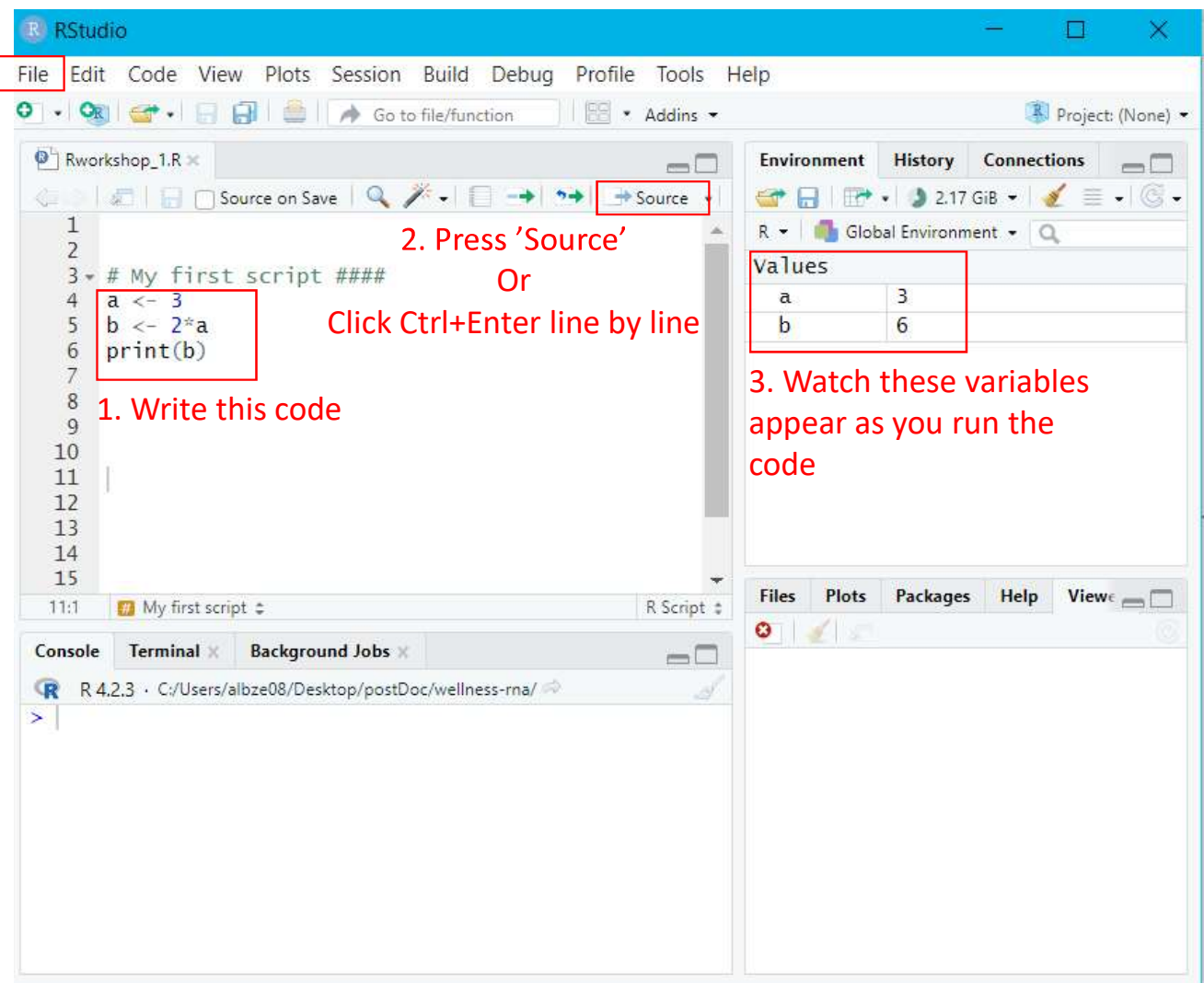


Introduction to R

Alberto Zenere

20 August 2025

Our first R script



Our first R script

1. Open Rstudio
2. Copy-paste the code you see on the left
3. Save (File -> Save as..)
4. Click 'Source'
5. Alternatively, go to line 4 and click Ctrl+Enter repeatedly

OBS. **print** is a command that shows the value of a variable (in this case of the variable *b*)

OBS. What you write after # is considered a comment

OBS. Save often while you code!

Our first R script

1. Type in ? And the name of the function you want to learn

2. Go to the Help section

3. Brief description of what the function does

4. Shows which options are available

5. Clarifies what each input does

6. Further description of the function

order (base)

Ordering Permutation

Description

`order` returns a permutation which rearranges its first argument into ascending or descending order, breaking ties by further arguments. `sort.list` does the same, using only one argument. See the examples for how to use these functions to sort data frames, etc.

Usage

```
order(..., na.last = TRUE, decreasing = FALSE,
      method = c("auto", "shell", "radix"))

sort.list(x, partial = NULL, na.last = TRUE, decreasing = FALSE,
         method = c("auto", "shell", "quick", "radix"))
```

Arguments

...	a sequence of numeric, complex, character or logical vectors, all of the same length, or a classed R object.
x	an atomic vector for methods "shell" and "quick". When x is a non-atomic R object, the default "auto" and "radix" methods may work if <code>order(x, ...)</code> does.
partial	vector of indices for partial sorting. (Non-NULL values are not implemented.)
decreasing	logical. Should the sort order be increasing or decreasing? For the "radix" method, this can be a vector of length equal to the number of arguments in ... and the elements are recycled as necessary. For the other methods, it must be length one.
na.last	for controlling the treatment of NAs. If TRUE, missing values in the data are put last; if FALSE, they are put first; if NA, they are removed (see 'Note').
method	the method to be used: partial matches are allowed. The default ("auto") implies "radix" for numeric vectors, integer vectors, logical vectors and factors with fewer than 2 ³¹ elements. Otherwise, it implies "shell". For details of methods "shell", "quick", and "radix", see the help for <code>sort</code> .

Details

In the case of ties in the first vector, values in the second are used to break the ties. If the values are still tied, values in the later arguments are used to break the tie (see the first example). The sort used is stable (except for method = "quick"), so any unresolved ties will be left in their original order.

How to create a variable

- Use "`<-`" to create a new variable, example: `a <- 3`
- "`=`" and "`<-`" are almost interchangeable, I suggest you stick to "`<-`"
- You can name variables however you want, except you cannot use special characters: `?`, `ä,ö,å`, `()`, `[]`, `{}`, `$`, `@`, `!`, etc.
- **The purpose of a variable name is to hint to its function** (e.g., `rna.count`, `input_file_name`, `number_genes`, etc.). Avoid generic names, like `"variable1"`, `"temp"`, etc.
- Be careful to not use variable names that are already function names, you would overwrite the function! Examples: **`print`**, **`sum`**, **`mean`**, **`median`**. Double check if a variable name is already used by simply typing it in the console.
- It is very common to use `"_"` or `"."` to combine multiple words into a single variable name. Examples: `"count.matrix"` or `"number_genes"`

Variables belong to different classes

- **Numeric**

```
number.samples <- 50
```

- **Logical**

```
is_gene_highly_expr <- TRUE
```

- **Character**

```
gene.name <- "STAT1" #notice that we encapsulate the word STAT1 between quotes
```

- **Multiple variables of the same type can be grouped into **vectors** using "c"**

```
number.samples <- c(24,55,66,3)  
is_gene_highly_expr <- c(TRUE, FALSE, FALSE, TRUE)  
gene.name <- c("STAT1", "LEP", "IL1")
```

- **Numbers can be organized in **matrices****

```
rna.count <- matrix(0,2,3) #this creates a 2x3 matrix full of zeros
```

Exercises

Exercise 1.a

To learn how functions work, we can use the command `?`

1. Write `?sum` in the console, then click Enter to learn about the command `sum`
2. Do the same for the commands `mean`, `median`, `seq`, `matrix`

Exercise 1.b

1. Create a variable that contains your year of birth
2. Create a variable that contains your current age
3. Using the first two, create a third variable that contains what year we are in today (using the function `sum`)
4. Use the command `print` to show its value

Exercises

Exercise 1.c

1. Create a vector containing five random numbers (you can choose)
2. Using the command **sum**, find the total sum of these numbers
3. Using the command **mean**, find the mean of these numbers
4. Using the command **median**, find the median of these numbers
5. Assign the value of the sum, mean and median to three variables and check their values in the workspace (top right in Rstudio)

Exercise 1.d

1. Create a variable which contains the numbers from 5 to 15 (use the command **:**)
2. Create a variable which contains the numbers from 5 to 15, hopping two by two (meaning 5,7,9,11,13,15; use the command **seq**)
3. Create a vector of 5 containing TRUE in all positions (use the command **rep**)
4. Create a vector of 5 containing "R is great" in all positions (use the command **rep**)
5. Create a 4x5 matrix containing all 1's (use the command **matrix**)

Exercises

Exercise 1.e

1. Create a numeric vector containing the elements 25, 43, 22, 31, 35
2. Calculate the mean and median of these numbers
3. Add a new elements, 50, to the vector and calculate the new mean and median

Exercise 1.f

1. Open ChatGTP (create and account if you haven't one)
2. Ask ChatGTP "Write a simple R exercise to practice on the vectors using basic functions"
3. Solve the exercise

Subsetting I/III

Very often we want to extract only part of a vector, this is called subsetting variables

Square brackets `[]` are used to subset

Example: subsetting a vector

```
patient.age <- c(18,23,5,45,34,17,9)
age.first <- patient.age[1] #extracts the first element
age.24 <- patient.age[2:4] #extracts elements 2,3,4
```

Example: subsetting a matrix

```
rand.mat <- matrix(rnorm(9),3,3) #creates a 3x3 matrix of random values
row.first <- rand.mat [1,] #extracts the entire first row
col.first <- rand.mat [,1] #extracts the entire first column
subset.mat <- rand.mat [1:2, 2:3] #extracts the sub-matrix formed by rows 1-2 and
columns 2-3
```

Subsetting II/III

Often we want to extract the elements that satisfy a certain condition, in this case we can use logical vectors

Example: extract the ages above or equal 18

```
patient.age <- c(18,23,5,45,34,17,9)
adult.patient <- (patient.age >= 18) #this is a logical vector
(TRUE, TRUE, FALSE, TRUE, TRUE, FALSE, FALSE)
print(adult.patient)
adult.ages <- patient.age[adult.patient]
print(adult.ages)
```

Example: extract the John's

```
patient.name <- c("John", "Alex", "Elsa", "John")
adult.john <- patient.name == "John" #this is a logical vector
print(adult.john)
```

OBS. Notice we use a double == to create logical vectors (while a single = is used to assign a value)

Subsetting III/III

Example: extract characters equal to " antibiotic"

```
patient.treatment <- c("placebo", "antibiotic", "antibiotic", "placebo")
patient.antibiotic <- patient.treatment == "antibiotic" #notice the double ==, which
we must use to create logical variables
print(patient.antibiotic)
number.antibiotic <- sum(patient.antibiotic) #the command sum can be used to count
the number of TRUE elements
print(number.antibiotic )
```

In case of a matrix, we may want to retain only some of the rows or columns

Example: extract the rows with a summed value above 1

```
rand.mat <- matrix(rnorm(12),3,4) #creates a 3x4 matrix of random values
print(rand.mat)
row.sum <- rowSums(rand.mat) #for each row, calculates the sum
col.sum <- colSums(rand.mat) #for each column, calculates the sum
selected.rows <- row.sum>1
subset.mat <- rand.mat[selected.rows, ]
print(subset.mat)
```

Exercises

Exercise 2.a

Consider two variables containing a group of genes and corresponding p-values:

```
gene <- c("STAT1", "LAT", "RAP", "LEP", "mTOR", "IL1")  
p <- c(0.01, 0.2, 0.004, 0.9, 0.7, 0.13)
```

1. Create a logical variable that indicates which p-values are below 0.05
2. Use this logical variable to extract the corresponding genes with a p-value below 0.05

Exercise 2.b

Consider a 4x5 matrix of random values (using **rnorm** as in the previous slide)

1. Subset the matrix to contain only rows have a sum below 0 (use **rowSums**)
2. Subset the matrix to contain only column have a sum below 0 (use **colSums**)

Exercises

Exercise 2.c

Create the following vector: `family_age <- c(Veronica=5, Gianni=11, Maria=34, Aldo=35)`. This is called a named vector since each element in the vector has a name.

1. Extract the 1st and 4th elements
2. Extract ages lower than 15
3. Extract the names of the vector using the function *names()*
4. Extract the age of Gianni using `family_age["Gianni"]`

Exercise 2.d

Create the following matrix:

```
temperature_week <- matrix( c(25,26,24, 14,12,16), nrow=3, ncol=2)
colnames(temperature_week) <- c("Greece", "Norway")
rownames(temperature_week) <- c("Monday", "Tuesday", "Wednesday")
```

1. Extract the 1st row
2. Extract the 2nd column
3. Extract temperatures related to "Greece"
4. Extract temperatures related to "Tuesday"
5. Calculate the mean of temperatures measured in the two countries

Common functions

Numerical	
sum, median, mean	(Seen before)
which.max / which.min	Position of the highest (or lowest) number
sort	Orders the vector from lowest to highest value (or the opposite, if use the option decreasing=TRUE)
seq	Create sequences of numbers
Character	
gsub	Replaces patterns of letters with a new pattern
strsplit	Splits the word at every occurrence of a specified patterns
Logical	
sum	Returns the number of TRUE elements
any	Returns TRUE if there is at least one TRUE element
all	Returns TRUE if all the elements are TRUE
which	Returns the position of the TRUE elements
Data.frame	
rowSums / colSums	Returns the sum of values in every row (or every column)
rowMeans / colMeans	Returns the mean of values in every row (or every column)

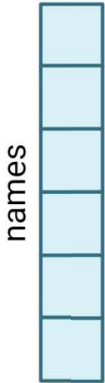
Further exercises

- <https://www.r-exercises.com/2019/07/29/creating-vectors/>
- <https://www.r-exercises.com/2019/08/05/working-with-vectors/>
- <https://www.r-exercises.com/2019/08/12/vectors-and-functions/>
- Anything ChatGTP comes up with

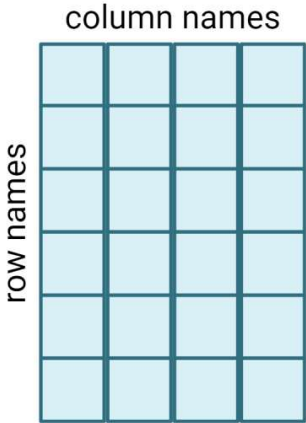
Overview of data structures



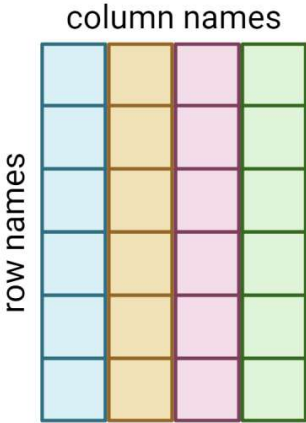
Basic element
The most common are
numeric, logical and
character



Vector
Multiple elements of the
same class
Created using `c()`, `rep()`, `seq()`



Matrix
Multiple columns that
contain the same data class
Created using `matrix()`



Data frame
Each column can contain
a different data class
Created using `data.frame()`

Functions for numerical vector	
sum, median, mean	(Seen before)
which.max / which.min	Position of the highest (or lowest) number
sort	Orders the vector from lowest to highest value (or the opposite, if use the option decreasing=TRUE)
seq	Create sequences of numbers
names	Used to set the names of each element (can be used for vectors of any class)
Functions for character vector	
gsub	Replaces patterns of letters with a new pattern
strsplit	Splits the word at every occurrence of a specified patterns
Functions for logical vector	
sum	Returns the number of TRUE elements
any	Returns TRUE if there is at least one TRUE element
all	Returns TRUE if all the elements are TRUE
which	Returns the position of the TRUE elements
Functions for data frames	
rowSums / colSums	Returns the sum of values in every row (or every column)
rowMeans / colMeans	Returns the mean of the values in each row (or each column)
rownames/colnames	Used to set the row and column names

Data frames

- We can store numbers, characters, logicals within the same variable using the function ***data.frame()***

Example: create a simple data.frame

```
df <- data.frame() # Initializes an empty data.frame
df <- data.frame(name=c("John", "Tom"), age=c(23,44)) # data.frame with two
columns and two rows
df$married <- c(FALSE,TRUE) # We can add a new column
print(df)
```

- We use the functions **rownames** and **colnames** to clarify what the data.frame contains, and also to subset

Example: use row names and column names

```
print(colnames(df)) #we have already specified the column names
rownames(df) <- df$name
print(df["John", ]) #prints the row with name "John"
print(df$name) #The symbol $ is used to extract a specific column
```

Exercises on data.frame I/III

Exercise 3.a: Create data frame containing gene counts and another containing information on the samples

In bioinformatics, we often work with two paired data.frame, one containing the gene counts and the second containing information on the samples

1. Create a 4x5 matrix of random elements using **rnorm**, call it `count.genes`
2. As row names, insert the names of 4 genes of your choice
3. As column names, use `sample1`, `sample2`, ..., `sample5`
4. Create a second data.frame of dimensions 5x3, call it `info.samples`
5. In the first column of `info.samples`, insert the names of 5 individuals
6. In the second column, set their ages
7. In the third column, specify if each individual belongs to the patient or control group (3 individuals in each group)

To summarize:

- the rows of `count.gene` correspond to genes, while the columns to samples.
- the rows of `info.samples` correspond to samples, while the columns to the information available.

Exercise 3.b: Subset a data frame

Using the two data frames you created in Exercise 3.a

1. Using a logical vector, extract the part of **both** data.frame corresponding to patients
2. Using a logical vector, extract the part of **both** data.frame with individuals older than the median age

Exercises on data.frame II/III

Exercise 3.c: Manipulate a data frame

1. Create a 4x5 matrix of random elements (use the functions **rnorm** and **matrix**)
2. Use **rownames** and **colnames** to assign row and column names
3. Convert the matrix into a data frame (use **as.data.frame**)
4. Add a new column called `sum_row` containing the sum of each row (use **rowSums**)
5. Add a new row containing `mean_col` containing the mean of each column (use **colMeans** and **rbind**)
6. [More difficult] Order the rows from highest to lowest mean. In other words, the first row should have the highest mean, and so on (use **order**).
7. [More difficult] Extract only the rows with a sum greater than zero (use **subset**).

Read text files

read.table is a common function to read text files.

There are many options, the most important are:

- `sep`: which separator is used in the text files (usually one between tab, space and comma)
- `header`: should we use the first line of the text file as column names?
- `row.names` (if any): position of the row names in the text file

OBS. It is crucial to set the working directory first using the function **setwd**

Example

Load the files `count_gene.txt` and `sample_info.txt` (you find them on Lisam → Course documents)

```
setwd("C:/Users/albze08/Desktop/teaching/2023_VT1_8BKG37") #change it to your folder!
count.gene <- read.table("gene_count.txt", sep=" ", header=T, row.names = 1)
sample.info <- read.table("sample_info.txt", sep="\t", header=T) #\t stands for Tab;
T stands for TRUE
```

OBS. in **setwd**, you need to use `"/"` and not `"\"`

Write text files

write.table is the most common function. It has many options, such as:

- `sep`: which separator to use in the text file
- `col.names`: should we write the column names in the text file?
- `row.names`: should we write the row names in the text file?
- `quote`: should characters be put between quotes (usually no)

Example

```
rand.mat <- matrix(rnorm(12), 3, 4)
colnames(rand.mat) <- c("sample1", "sample2", "sample3", "sample4")
write.table(rand.mat, file="rand_matrix.txt", row.names=F, col.names=T,
sep=";", quote=F) #try to see what happens with row.names=T
```

Exercises

Exercise 4.a

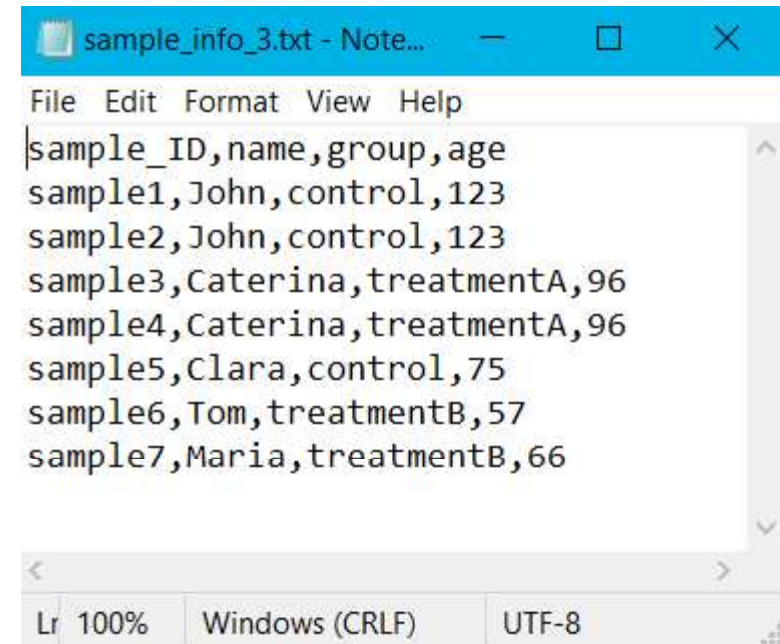
Read in the file `sample_info.txt`

Multiply the column `age` by 3

Save the results in a new text file where:

- There are no quotation marks
- The words are separated by a comma
- The option `row.names` is set to `FALSE`

Should look
like this



```
sample_ID,name,group,age
sample1,John,control,123
sample2,John,control,123
sample3,Caterina,treatmentA,96
sample4,Caterina,treatmentA,96
sample5,Clara,control,75
sample6,Tom,treatmentB,57
sample7,Maria,treatmentB,66
```

Exercise 4.b

Same as **Exercise 4.a** but use different options choices to see what happens

Exercises on data.frame III/III

Exercise 3.d: Manipulate two data frame

Read in the file `sample_info.txt` and `gene_count.txt`, notice that the rows of `sample_info.txt` correspond to the columns of `gene_count.txt`

1. Extract the gene counts corresponding to controls
2. Calculate the mean gene expression of John (should be a vector of 5 elements); as names for this vector use the row names of the gene count data frame.
3. Create a named vector of three elements containing the mean expression of geneC in controls, treatmentA and treatmentB.

Methodic approach to fix errors

You will make plenty of mistakes all the time (we all do) → How to fix them?

1. Read the error message carefully, (a) it points to the problematic line and (b) gives hints on the cause
2. If still unclear, go to the specific line and double check that each variable contains the value what you expect (correct dimensions, no missing values, etc.)
3. If it concerns a particular function, look at its Help section to make sure the arguments you are giving are correct
4. Patience is your best friend

Common errors

"object not found" usually because of a typo or because you did not run part of the code

"cannot open a file" usually because of typos in the file name or because you did not set the working directory

"subscript out of bounds" you are trying to subset an object, but the dimensions do not match. For example, you may be trying to access the 5th row of a data frame with only 4 rows.

"could not find function" typos in the function name or you forgot to load the corresponding package

Exercises

Each exercise contains code with an error, find it and correct it.

Exercise 5.a

```
rand.mat <- matrix(rnorm(12), 4,3)
colnames(rand.mat) <- c("sample1", "sample2", "sample3")
sample1.count <- rand.mat$sample1
```

Exercise 5.b

```
rand.mat <- matrix(rnorm(12), 4,3)
idx <- c(F, T, T, T, F)
subset.mat <- rand.mat [idx,]
```

Exercise 5.c

```
rand.mat <- matrix(rnorm(12), 4,3)
idx <- rowSums(rand.mat)>0
subset.mat <- rand.mat [,idx]
```

Exercises

Exercise 5.d

```
df <- data.frame(city=c("Paris", "Rome", "New York" ),  
inhabitants=c(2.2,2.9,8.5))  
most.inhabitants <- max(df$inhabtians)
```

Exercise 5.e

```
df <- data.frame(city=c("Paris", "Rome", "New York" ),  
inhabitants=c(2.2,2.9,8.5))  
city.5 <- nth(df$city, 5)
```

Exercise 5.f

```
df <- data.frame( Name = c("Alice", "Bob", "Charlie", "David", "Eve"),  
Age = c(23, 30, 22, 40),  
Score = c(76, 95, 78, 85, 92) )
```

Exercises

Exercise 5.g

Correct the data frame in 5.f, then:

```
mean_score <- colMeans(df)
average_age <- mean(df$Ages)
sorted_df <- df[order(df$Score, decreasing=True), ]
older_than_25 <- subset(df, Age > 25 & Passed == "TRUE")
```

Exercise 5.h

Ask ChatGTP to "write R code snippets with intentional errors to find and correct".

The **if/else** statement

Used to run a part of code only if a condition is satisfied

Example

Generate a random number. If the number is positive, print it.

```
rand.num <- rnorm(1)
if (rand.num>0) {
    print(rand.num)
}
```

Example

Generate a random number. If the number is positive, print it; otherwise, print the message "The number is negative"

```
rand.num <- rnorm(1)
if (rand.num>0) {
    print(rand.num)
} else {
    print("The number is negative")
}
```

The **for** command I/II

Used to repeat the same lines multiple times

Example. For loop over a numeric vector

Print all numbers from 1 to 10.

```
seq.num <- 1:10  
for (n in seq.num) {  
  print(n)  
}
```

Everything inside the **for** loop will be repeated for every value of `n`, which is specified by `seq.num`

The **for** command II/II

Example. For loop over each row of a data.frame

Assume we want to double the column age in the data frame. Usually we would do `df$age <- df$age*2` but, just for the sake of practicing for loops, we can also do it the following way:

```
df <- read.table("sample_info.txt", sep="\t", header=T)
seq.num <- 1:nrow(df) #nrow() return the number of rows of a matrix or data.frame
for (n in seq.num){ #We use the variable n to go through every row of the data frame
  age.2 <- df$age[n]*2 #n take the values 1,2,3,...,nrow(df)
  df$age[n] <- age.2
}
```

Example. For loop over some rows of a data.frame

Assume we want to double only the ages in the even rows of the data.frame.

```
df <- read.table("sample_info.txt", sep="\t", header=T)
seq.num <- seq(from=2, to=nrow(df), by=2)
for (n in seq.num){ #we change seq.num to contain only even numbers
  age.2 <- df$age[n]*2 #n take the values 2,4,6,...,nrow(df)
  df$age[n] <- age.2
}
```

Exercises

Exercise 5.a

- Create a for loop that prints the square root of the numbers from 20 to 30

Exercise 5.b

- Create a vector of 10 random numbers
- Create a for loop to print its elements one by one

Exercise 5.c

- Create a 4x5 matrix of random numbers
- For each row, print the mean of the row
- For each column, print the sum of the column

Exercises

Exercise 5.d

- Create a vector of 10 random numbers using **rnorm**
- Create a for loop that, for each of its elements:
 - prints the `log2()` of the number if positive (use function **log2**)
 - prints "Cannot calculate the log of negative numbers", if negative

Exercise 5.e

Load `sample_info.txt` into a variable called `df`. Modify it in the following way:

- If the name is John, then add 5 to the age
- If group is control, then change it to healthy (Hint: use the function **gsub**)

Exercises

Exercise 5.f: **for** loop containing **if** statements

Load `sample_info.txt` and `gene_count.txt`. Using if statements and for loops:

- Add a new column to `sample_info` to indicate if the mean expression of each gene is above 5.3

Exercise 5.g: **for** loop containing **if** statements

Create a vector of 10 random numbers, using `rnorm`. For each of these numbers:

- if greater than 10, multiply by 2 and print
- If lesser or equal to 10, add 5 and print

Exercise 5.h: Manipulate two data frame

Read in the file `sample_info.txt` and `gene_count.txt`

Remember that the rows of `sample_info.txt` correspond to the columns of `gene_count.txt`

1. For each gene, calculate difference in mean expression between controls and treatmentA

Further exercises

- <https://leetcode.com/problemset/>